Diário de Bordo — Aula 9

Do Modelo Lógico ao Banco de Dados Relacional: DDL, Chaves, Restrições e Testes de Integridade

Data	Aula de 15/10/2026 — preenchido em 26/10/2026, após a aula
Grupo	7
Empresa / projeto ERP	Petly Club — ERP para pet shop com loja física e loja virtual
Responsável pelo preenchimento	Gabriela Colombo

> **Observação:** este diário não foi entregue no dia da aula. Ele foi preenchido em 26/10/2026. O script e os testes abaixo foram executados nessa data no PostgreSQL 16, e as mensagens de erro foram copiadas da saída do SGBD sem alteração.

Missão da aula

Transformar o modelo lógico do recorte do ERP em um script DDL executável, com PK, FK, NOT NULL, UNIQUE e CHECK, e validar o banco com testes de rejeição.

Checklist de verificação da missão

☑ Recorte com pelo menos 4 tabelas relacionadas (6 tabelas).
☑ Pelo menos uma relação N:N resolvida por tabela associativa (ITEM_PEDIDO).
☑ Script DDL (.sql) contendo PK, FK, NOT NULL, UNIQUE e CHECK.
☑ Pelo menos 4 testes de violação/rejeição executados no SGBD (6 comandos).
☑ Mapeamento claro: REGRA DE NEGÓCIO → MODELO LÓGICO → RESTRIÇÃO → DDL → TESTE.

Produto: script `recorte_aula9.sql` + testes `testes_aula9.sql` + este diário.

Missão 1 — Escolha e validação do recorte

Processo escolhido pelo grupo: venda (P05 e P06 — venda na loja física e na loja virtual): CLIENTE → PEDIDO → ITEM_PEDIDO ← PRODUTO

Tabelas presentes no recorte:

1. CLIENTE (tabela independente)
2. CATEGORIA e FUNCIONÁRIO (tabelas independentes; entram porque PRODUTO e PEDIDO têm FK para elas)
3. PRODUTO e PEDIDO (tabelas dependentes)
4. ITEM_PEDIDO (tabela associativa N:N entre PEDIDO e PRODUTO)

Missão 2 — Mapeamento de regras de negócio e restrições

Tabela / coluna	Restrição técnica	Regra de negócio que protege
CLIENTE (cpf)	UNIQUE + NOT NULL + CHECK (11 dígitos)	Só compra quem tem cadastro, e o mesmo CPF não pode ser cadastrado duas vezes (RN02, RN18).
PRODUTO (preco)	NOT NULL + CHECK (preco > 0)	Todo produto à venda tem preço, e ele não pode ser zero nem negativo (RP04).

Tabela / coluna	Restrição técnica	Regra de negócio que protege
PRODUTO (id_categoria)	FK + NOT NULL	Todo produto pertence a uma categoria existente (RN04).
PRODUTO (codigo_barras)	UNIQUE	O código de barras localiza um único produto no caixa (RNF05).
PEDIDO (id_cliente)	FK + NOT NULL	Todo pedido pertence a um cliente cadastrado (RN01, RP02).
PEDIDO (canal_venda, tipo_recebimento, status)	CHECK (lista de valores)	Só existem os dois canais (RN05), as três formas de recebimento (RN17) e os status definidos (RN16).
PEDIDO (canal_venda, id_funcionario)	CHECK (loja física exige funcionário)	Toda venda da loja física é registrada por um funcionário (RN14).
ITEM_PEDIDO (id_pedido, id_produto)	PK composta (as duas também FK)	O mesmo produto aparece uma única vez em cada pedido, com a quantidade total.
ITEM_PEDIDO (quantidade)	CHECK (quantidade > 0)	Não existe item vendido com quantidade zero ou negativa (RN03).
ITEM_PEDIDO (preco_unitario)	NOT NULL + CHECK (> 0)	O preço praticado na venda fica registrado, mesmo que o preço do produto mude depois (RN13).
FUNCIONÁRIO (login)	UNIQUE + NOT NULL	Cada funcionário tem um acesso próprio ao sistema (RP08).

Missão 3 — Script DDL do recorte (PostgreSQL 16)

```sql
-- 1. TABELAS INDEPENDENTES

CREATE TABLE categoria (
    id_categoria   SERIAL        PRIMARY KEY,
    nome           VARCHAR(60)   NOT NULL UNIQUE,
    descricao      VARCHAR(255)
);

CREATE TABLE cliente (
    id_cliente     SERIAL        PRIMARY KEY,
    nome           VARCHAR(120)  NOT NULL,
    cpf            CHAR(11)      NOT NULL UNIQUE,                -- RN18
    telefone       VARCHAR(15),
    email          VARCHAR(120),
    CONSTRAINT ck_cliente_cpf CHECK (cpf ~ '^[0-9]{11}$')
);

CREATE TABLE funcionario (
    id_funcionario SERIAL        PRIMARY KEY,
    nome           VARCHAR(120)  NOT NULL,
    cpf            CHAR(11)      NOT NULL UNIQUE,
    cargo          VARCHAR(30)   NOT NULL,
    login          VARCHAR(40)   NOT NULL UNIQUE,
    senha_hash     VARCHAR(255)  NOT NULL,                       -- RNF02
    CONSTRAINT ck_funcionario_cargo CHECK
        (cargo IN ('atendente', 'operador de caixa', 'estoquista', 'gerente'))  -- RP08
);
```

```sql
-- 2. TABELAS DEPENDENTES E ASSOCIATIVAS (N:N)


CREATE TABLE produto (
    id_produto        SERIAL        PRIMARY KEY,
    nome              VARCHAR(120)  NOT NULL,
    descricao         VARCHAR(255),
    marca             VARCHAR(60)   NOT NULL,
    codigo_barras     VARCHAR(14)   UNIQUE,                       -- RNF05
    preco             NUMERIC(10,2) NOT NULL,
    estoque_minimo    INTEGER       NOT NULL DEFAULT 0,           -- RP03
    controla_validade BOOLEAN       NOT NULL DEFAULT FALSE,       -- RN11
    id_categoria      INTEGER       NOT NULL,                     -- RN04
    CONSTRAINT ck_produto_preco   CHECK (preco > 0),
    CONSTRAINT ck_produto_minimo  CHECK (estoque_minimo >= 0),
    CONSTRAINT fk_produto_categoria FOREIGN KEY (id_categoria)
        REFERENCES categoria (id_categoria)
);

CREATE TABLE pedido (
    id_pedido        SERIAL        PRIMARY KEY,
    data_pedido      TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    canal_venda      VARCHAR(15)   NOT NULL,                     -- RN05
    tipo_recebimento VARCHAR(20)   NOT NULL,                     -- RN17
    status           VARCHAR(20)   NOT NULL DEFAULT 'pendente',  -- RN16
    desconto         NUMERIC(10,2) NOT NULL DEFAULT 0,
    id_cliente       INTEGER       NOT NULL,                     -- RN01, RP02
    id_funcionario   INTEGER,                                    -- RN14
    CONSTRAINT ck_pedido_canal  CHECK (canal_venda IN ('loja fisica', 'loja virtual')),
    CONSTRAINT ck_pedido_receb  CHECK (tipo_recebimento IN ('balcao', 'retirada na loja',
'entrega')),
    CONSTRAINT ck_pedido_status CHECK (status IN ('pendente', 'pago', 'em preparacao',
                                      'enviado', 'entregue', 'retirado', 'cancelado')),
    CONSTRAINT ck_pedido_desconto CHECK (desconto >= 0),
    -- venda no balcão só existe na loja física, e a loja física sempre tem funcionário
    CONSTRAINT ck_pedido_balcao CHECK (tipo_recebimento <> 'balcao' OR canal_venda = 'loja
fisica'),
    CONSTRAINT ck_pedido_func   CHECK (canal_venda <> 'loja fisica' OR id_funcionario IS NOT
NULL),
    CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)
        REFERENCES cliente (id_cliente),
    CONSTRAINT fk_pedido_funcionario FOREIGN KEY (id_funcionario)
        REFERENCES funcionario (id_funcionario)
);

CREATE TABLE item_pedido (
    id_pedido      INTEGER       NOT NULL,
    id_produto     INTEGER       NOT NULL,
    quantidade     INTEGER       NOT NULL,
    preco_unitario NUMERIC(10,2) NOT NULL,                       -- RN13
    CONSTRAINT pk_item_pedido PRIMARY KEY (id_pedido, id_produto),
    CONSTRAINT ck_item_qtd    CHECK (quantidade > 0),
    CONSTRAINT ck_item_preco  CHECK (preco_unitario > 0),
    CONSTRAINT fk_item_pedido  FOREIGN KEY (id_pedido)
        REFERENCES pedido (id_pedido) ON DELETE CASCADE,
    CONSTRAINT fk_item_produto FOREIGN KEY (id_produto)
        REFERENCES produto (id_produto)
);
```

Dados válidos inseridos antes dos testes (todos aceitos pelo SGBD):

```sql
INSERT INTO categoria (nome, descricao) VALUES
    ('Alimentacao', 'Racoes e petiscos'),
    ('Higiene', 'Shampoos, tapetes higienicos e areias');

INSERT INTO cliente (nome, cpf, telefone, email) VALUES
```

```
            ('Maria Souza', '12345678901', '11988887777', 'maria@email.com'),
            ('Joao Lima',   '98765432100', NULL,          'joao@email.com');

        INSERT INTO funcionario (nome, cpf, cargo, login, senha_hash) VALUES
            ('Ana Caixa', '11122233344', 'operador de caixa', 'ana.caixa', '$2b$12$exemplohash');

        INSERT INTO produto (nome, marca, codigo_barras, preco, estoque_minimo, controla_validade,
id_categoria) VALUES
            ('Racao para Cao 15 kg', 'Marca A', '7890000000011', 189.90, 5, TRUE,  1),
            ('Shampoo Neutro 500 ml', 'Marca B', '7890000000028',  29.90, 10, TRUE, 2);

        INSERT INTO pedido (canal_venda, tipo_recebimento, status, id_cliente, id_funcionario) VALUES
            ('loja fisica',  'balcao',  'pago',     1, 1),
            ('loja virtual', 'entrega', 'pendente', 2, NULL);

        INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES
            (1, 1, 2, 189.90),
            (1, 2, 1,  29.90),
            (2, 1, 1, 189.90);
```

Missão 4 — Bateria de testes de integridade

[TESTE 01] Violação de unicidade (UNIQUE)

Comando executado:

```
        INSERT INTO cliente (nome, cpf, email)
        VALUES ('Maria S. Copia', '12345678901', 'outra@email.com');
```

Erro retornado pelo SGBD:

```
ERROR:  duplicate key value violates unique constraint "cliente_cpf_key"
DETAIL:  Key (cpf)=(12345678901) already exists.
```

O SGBD rejeitou corretamente? ☑ Sim ☐ Não — o CPF já pertence à cliente Maria Souza (RN18).

[TESTE 02] Violação de integridade referencial (FOREIGN KEY inexistente)

Comando executado:

```
        INSERT INTO pedido (canal_venda, tipo_recebimento, id_cliente)
        VALUES ('loja virtual', 'entrega', 999);
```

Erro retornado pelo SGBD:

```
ERROR:  insert or update on table "pedido" violates foreign key constraint "fk_pedido_cliente"
DETAIL:  Key (id_cliente)=(999) is not present in table "cliente".
```

O SGBD rejeitou corretamente? ☑ Sim ☐ Não — não existe cliente 999, e não há pedido sem cliente cadastrado (RN01, RP02).

[TESTE 03] Violação de validação de domínio (CHECK ou NOT NULL)

03a — CHECK de preço

Comando executado:

```
        INSERT INTO produto (nome, marca, preco, id_categoria)
        VALUES ('Petisco Dental', 'Marca C', 0, 1);
```

Erro retornado pelo SGBD:

```
ERROR:  new row for relation "produto" violates check constraint "ck_produto_preco"
DETAIL:  Failing row contains (3, Petisco Dental, null, Marca C, null, 0.00, 0, f, 1).
```

O SGBD rejeitou corretamente? ☑ Sim ☐ Não — produto com preço zero.

03b — CHECK de regra do negócio

Comando executado:

```
        INSERT INTO pedido (canal_venda, tipo_recebimento, id_cliente)
        VALUES ('loja fisica', 'balcao', 1);
```

Erro retornado pelo SGBD:

```
ERROR:  new row for relation "pedido" violates check constraint "ck_pedido_func"
DETAIL:  Failing row contains (4, 2026-10-26 16:23:16.448712, loja fisica, balcao, pendente, 0.00, 1,
null).
```

O SGBD rejeitou corretamente? ☑ Sim ☐ Não — venda na loja física sem funcionário responsável (RN14).

03c — NOT NULL

Comando executado:

```
        INSERT INTO produto (nome, marca, preco)
        VALUES ('Coleira P', 'Marca D', 39.90);
```

Erro retornado pelo SGBD:

```
ERROR:  null value in column "id_categoria" of relation "produto" violates not-null constraint
DETAIL:  Failing row contains (4, Coleira P, null, Marca D, null, 39.90, 0, f, null).
```

O SGBD rejeitou corretamente? ☑ Sim ☐ Não — produto sem categoria (RN04).

[TESTE 04] Violação da regra da tabela associativa N:N (PK composta)

Comando executado:

```
        INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario)
        VALUES (1, 1, 3, 189.90);
```

Erro retornado pelo SGBD:

```
ERROR:  duplicate key value violates unique constraint "pk_item_pedido"
DETAIL:  Key (id_pedido, id_produto)=(1, 1) already exists.
```

O SGBD rejeitou corretamente? ☑ Sim ☐ Não — a Ração para Cão 15 kg já é um item do pedido 1. Para vender mais unidades, a quantidade do item existente é alterada.

Conferência depois dos testes: SELECT `count(*)` em cada tabela retornou 2 clientes, 2 pedidos, 2 produtos e 3 itens, os mesmos números de antes dos testes. Nenhuma linha inválida foi gravada.

Defenda sua modelagem e DDL

1. De onde veio a necessidade de cada restrição CHECK/UNIQUE definida no DDL?
Cada uma vem de uma regra de negócio ou requisito documentado na primeira entrega (tabela da Missão 2). O UNIQUE do CPF vem da RN18 e da exigência de cadastro do modelo inspirado no Sam's Club. Os CHECKs de canal, recebimento e status vêm das listas fechadas definidas em RN05, RN17 e RN16. O CHECK de preço e quantidade positivos evita valores que não têm sentido numa venda. O

CHECK que exige funcionário na loja física vem da RN14. Não foi criada nenhuma restrição sem regra que a justifique.

2. Como o DDL da tabela associativa resolveu a relação N:N do recorte?
ITEM_PEDIDO tem duas FKs, uma para PEDIDO e outra para PRODUTO, e a PK é formada pelas duas (PRIMARY KEY (id_pedido, id_produto)). Com isso, o N:N vira dois 1:N: um pedido tem vários itens e um produto aparece em vários itens. A PK composta impede o mesmo produto duas vezes no pedido (Teste 04). A associativa guarda os atributos da ligação, quantidade e preco_unitario. Com ON DELETE CASCADE na FK do pedido, os itens não ficam órfãos se um pedido for excluído; já a FK do produto não tem cascade, então não é possível excluir um produto que já foi vendido.

3. Qual é o limite das restrições do SGBD que o grupo identificou?
CHECK só enxerga a própria linha, e FK só verifica se a chave existe. Por isso, algumas regras do Petly Club não podem ser garantidas só com constraints:

- não vender mais do que a quantidade disponível do lote (RN07), porque depende da soma das movimentações em outra tabela;
- vender primeiro do lote que vence antes (RN20) e bloquear lote vencido (RF12);
- e-mail obrigatório só para quem compra na loja virtual, porque o canal está em PEDIDO e o e-mail em CLIENTE;
- duas vendas simultâneas da última unidade (RNF10).

Essas regras precisam de transação, trigger ou validação na aplicação. Isso entra no script completo da entrega final.

Registro do trabalho e dificuldades do grupo

Síntese do que foi desenvolvido: script DDL do recorte de venda (6 tabelas) com PK, FK, PK composta, NOT NULL, UNIQUE, CHECK e DEFAULT, carga mínima de dados válidos e bateria de 6 testes de rejeição, todos recusados pelo SGBD.

Dificuldades encontradas e correções realizadas no modelo lógico/DDL:

- O recorte “Cliente → Pedido → Item_Pedido → Produto” não roda sozinho, porque PRODUTO tem FK para CATEGORIA e PEDIDO para FUNCIONÁRIO. As duas tabelas foram incluídas e criadas antes, na ordem independentes → dependentes → associativa.
- Subtotal e valor total não viraram colunas, porque são derivados (normalização da aula 8). Eles serão calculados nas consultas.
- A senha do funcionário foi definida como senha_hash, para guardar só o hash (RNF02).
- Para a regra “loja física exige funcionário”, o grupo usou um CHECK entre duas colunas da mesma linha, em vez de deixar id_funcionario sempre obrigatório, porque na loja virtual o próprio cliente faz o pedido.

Contribuição dos integrantes

Nome do integrante	Contribuição realizada	Participou da 2ª parte?
		☐ Sim ☐ Não
		☐ Sim ☐ Não
		☐ Sim ☐ Não
		☐ Sim ☐ Não
		☐ Sim ☐ Não
		☐ Sim ☐ Não
		☐ Sim ☐ Não

Nome do integrante	Contribuição realizada	Participou da 2ª parte?
		☐ Sim ☐ Não
		☐ Sim ☐ Não
		☐ Sim ☐ Não

Registro de ausências / não participação